

# Dziedziczenie

Zbigniew Marszalek

# Dziedziczenie

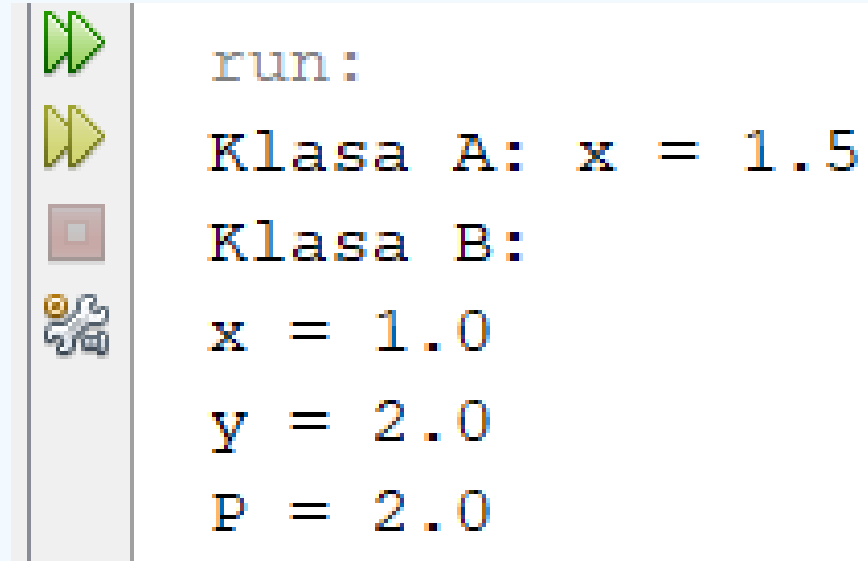
Jednym z istotnych elementów programowania obiektowego jest dziedziczenie umożliwiające tworzenie klasyfikacji nieracicznej. Ideą jest utworzenie ogólnej klasy definiującej właściwości wspólne dla pewnego zbioru klas. Z klasy głównej wywodzi się klasy pochodne posiadające pewne cechy unikatowe. W terminologii Java klasa, po której się dziedziczy, nosi nazwę klasy bazowej lub nadrzędnej. Klasa, która dziedziczy, nazywa się klasą pochodną lub podklasą. Podklasa dziedziczy wszystkie zmienne i metody klasy bazowej.

Deklaracja klasy, która dziedziczy po innej klasie ma postać:

```
class nazwa_podklasy extends nazwa_klasy_bazowe { }
```

```
class KlasaA{
    double x;
    void drukujx() {
        System.out.println("x = " + x);
    }
}
class KlasaB extends KlasaA{
    double y;
    void drukujy() {
        System.out.println("y = " + y);
    }
    void drukujp() {
        System.out.println("P = " + x*y);
    }
}
```

```
public class C2a {  
    public static void main(String[] args) {  
        KlasaA a = new KlasaA();  
        KlasaB b = new KlasaB();  
        // Klasę bazową można incjować niezależnie.  
        a.x = 1.5;  
        System.out.print("Klasa A: ");  
        a.drukujx();  
        b.x = 1.0;  
        b.y = 2.0;  
        System.out.println("Klasa B:");  
        b.drukujx();  
        b.drukujy();  
        b.drukujp();  
    }  
}
```



```
run:
Klasa A: x = 1.5
Klasa B:
x = 1.0
y = 2.0
P = 2.0
```

Podklasa Klasa B zawiera wszystkie składowe klasy bazowe A. Dlatego metoda Klasy B potrafi pobrać wszystkie wartości klasy bazowej i obliczyć iloczyn zmiennych. Innymi słowy składowe Klasy A są jednocześnie składowymi Klasy B. Klasa B dziedziczy całą charakterystykę klasy A i dodatkowo dodaje jedną zmienną. Nowa klasa nie musi odtwarzać wszystkich cech klasy bazowej. Po prostu rozszerza ją, by sprostać nowym wymaganiom.

# Dostęp do składowych a dziedziczenie

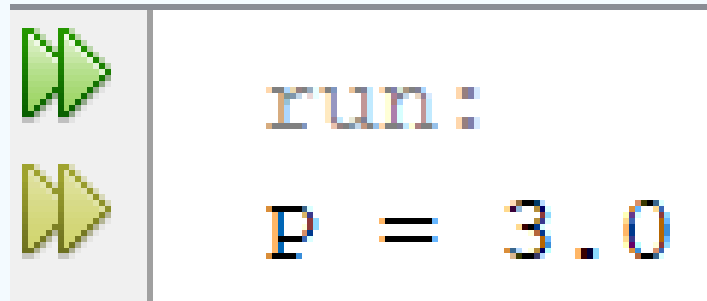
Choć klasa pochodna zawiera wszystkie składowe klasy bazowej, to nie ma dostępu do składowych klasy, które w klasie bazowej zostały oznaczone modyfikatorem *private*. Rozwiązaniem jest użycie modyfikatora *protected*, który daje dostęp do tych zasobów klasie potomnej np.:

```
class KlasaA{  
    protected double x, y;  
    void setxy(double x, double y) {  
        this.x = x;  
        this.y = y;  
    }  
}
```

```
class KlasaB extends KlasaA{
    double p;
    void Pole() {
        p = x*y;
    }
}

public class C3b {
    public static void main(String[] args) {
        KlasaB b = new KlasaB();
        b.setxy(1.5, 2.0);
        b.Pole();
        System.out.println("P = " + b.p);
    }
}
```

# Działanie programu



UWAGA: Składowe klasy zadeklarowane jako *private* pozostają prywatne nawet po dokonaniu dziedziczenia. Innymi słowy, nie są dostępne dla kodu spoza klasy, nawet jeśli jest to klasa pochodna.

# Słowo kluczowe *super*

W przypadku, gdy pola w klasie bazowej zostały zadeklarowane jako *private*, klasa potomna nie ma do nich dostępu. Nie może nadać im wartości, ani też ich odczytać. Aby zainicjować wartości w polach klasy bazowej w takim przypadku możemy wołać konstruktory klasy bazowej za pomocą słowa kluczowego *super*. Java potrafi sama dopasować odpowiedni konstruktor o potrzebnej liczbie parametrów tak jak to zostało pokazane w przykładnie.

```
class KlasaA{
    private double x, y;
    KlasaA(double x, double y) {
        this.x = x;
        this.y = y;
    }
    KlasaA(double xx) {
        x = y = xx;
    }
    KlasaA() {}
    double pole() {
        return x*y;
    }
}
```

```
class KlasaB extends KlasaA{  
    private double z;  
    KlasaB(double x, double y, double z) {  
        super(x,y) ;  
        this.z = z;  
    }  
    KlasaB(double xx) {  
        super(xx) ;  
        this.z = xx;  
    }  
    KlasaB() {}  
    double objentosc() {  
        return super.pole() *z;  
    }  
}
```

```
public class C3c {  
    public static void main(String[] args) {  
        KlasaB b1 = new KlasaB(1.0,2.0,3.0);  
        double o1 = b1.objentosc();  
        System.out.println("Objętość1 = " + o1);  
        KlasaB b2 = new KlasaB(3.0);  
        double o2 = b2.objentosc();  
        System.out.println("Objętość2 = " + o2);  
    }  
}
```



run:



Objętość1 = 6.0



Objętość2 = 27.0

# Dostęp do pól klasy bazowej za pomocą słowa kluczowego *super*

Za pomocą słowa kluczowego *super* można otrzymać dostęp do pól klasy bazowej z klasy potomnej. W takim przypadku słowo kluczowe *super* działa podobnie jak referencja *this* w odniesieniu do klasy nadrzędnej. Składnia referencji *super* jest następująca:

*super.składowa*

Element *składowa* może być metodą albo zmienną składową. Elementy do których odnosi się referencja *super* muszą mieć zadeklarowane prawo dostępu *protected* albo *public*.

```
class KlasaA{
    protected double x;
}
class KlasaB extends KlasaA{
    protected double x;
    KlasaB(double x1, double x2) {
        super.x = x1;
        x = x2;
    }
    void drukuj() {
        System.out.println("Z kalsy bazowej:"
                           + super.x);
        System.out.println("Z podkalsy:" + x);
    }
}
```

```
public class C3d {  
    public static void main(String[] args) {  
        KlasaB b = new KlasaB(1.0,2.0);  
        b.drukuj();  
    }  
}
```

|                                                                                   |                     |
|-----------------------------------------------------------------------------------|---------------------|
|  | run:                |
|  | Z kalsy bazowej:1.0 |
|  | Z podkalsy:2.0      |

W języku Java można tworzyć hierarchie wielopoziomowe z dowolnej liczby poziomów dziedziczenia. Podklasa może stanowić klasę bazową innej podklasy.

# Przysłanianie metod

Jeżeli w hierarchii klas metoda podklasy ma taką samą nazwę i sygnaturę typu jak metoda klasy bazowej, wtedy mówimy, że metoda podklasy przysłania metodę klasy bazowej. Wywołanie metody dla egzemplarza podklasy spowoduje wywołanie wersji zdefiniowanej w podklasie.

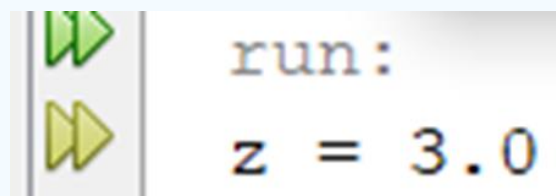
Przysłanianie metod w języku Java jest przykładem polimorfizmu w trakcie wykonywania programu. Polimorfizm stanowi podstawę programowania obiektowego, ponieważ umożliwia określenie w ogólnej klasie ogólnych metod wspólnych dla pewnego zbioru klas, a następnie, dzięki dziedziczeniu, zdefiniowanie w każdej z podklas implementacji tych metod wg. Zasady „jeden interfejs, wiele metod”.

```
class KlasaA{
    protected double x, y;
    KlasaA() {}
    KlasaA(double x, double y) {
        this.x = x;
        this.y = y;
    }
    void drukuj() {
        System.out.println("x = " + x) ;
        System.out.println("y = " + y) ;
    }
}
```

```
class KlasaB extends KlasaA{  
    protected double z;  
    KlasaB() {}  
    KlasaB(double x, double y, double z) {  
        super(x,y) ;  
        this.z = z;  
    }  
    void drukuj() {  
        System.out.println("z = " + z) ;  
    }  
}
```

# Przysłanianie metod

```
public class C3e {  
    public static void main(String[] args) {  
        KlasaB b = new KlasaB(1.0, 2.0, 3.0);  
        b.drukuj();  
    }  
}
```



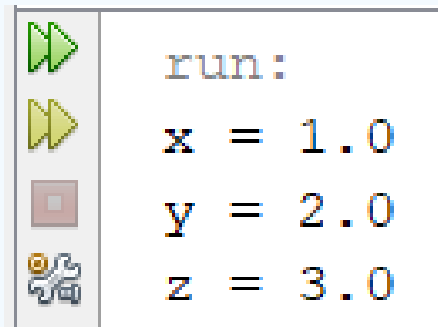
The screenshot shows a Java IDE interface. On the left, there are two green right-pointing arrows, which are standard icons for running or debugging code. To the right of these arrows, the text "run:" is displayed in a monospaced font. Below "run:", the output "z = 3.0" is shown, indicating the result of a calculation or a variable's value.

Przysłonięcie metody następuje tylko wtedy, gdy sygnatury typów obu metod są identyczne. Jeśli nie są identyczne, następuje zwykłe przeciążenie metod.

# Wywołanie przysłoniętej metody

Aby wywołać przysłoniętą metodę z klasy bazowej należy jawnie dokonać jej wywołania za pomocą polecenia `super` w metodzie przysłaniającej.

```
void drukuj () {  
    super.drukuj () ;  
    System.out.println ("z = " + z) ;  
}
```



# Klasy abstrakcyjne

Bazowa klasa abstrakcyjna określa strukturę dla klas potomnych i nie implementuje metod potomnych. Można powiedzieć, że klasa abstrakcyjna jest interfejsem dla klas wywodzących się z klasy bazowej. Poniżej umieszczono przykład klasy abstrakcyjnej.

```
package c3f;  
abstract class Pracownik {  
    protected int id;  
    protected String nazwisko, adres;  
    Pracownik() {}  
}
```

```
Pracownik(int id,String nazwisko,  
           String adres){  
    this.id = id;  
    this.nazwisko = nazwisko;  
    this.adres = adres;  
}  
public abstract double placa();  
// Metoda abstrakcyjna.  
}  
class Wynagrodzenie extends Pracownik {  
    private double placa;    // Wynagrodzenie  
    Wynagrodzenie(){};
```

```
Wynagrodzenie(int id, String nazwisko,  
               String adres, double placa) {  
    this.id = id;  
    this.nazwisko = nazwisko;  
    this.adres = adres;  
    this.placa = placa;  
}  
public double placa() {  
    System.out.println  
        ("Wynagrodzenie dla pracownika " + nazwisko );  
    System.out.println("wynosi " + placa );  
    return placa;  
}  
}
```

```
public class C3f {  
    public static void main(String[] args) {  
        Wynagrodzenie s = new Wynagrodzenie  
            (1, "Kowalski", "Katowice", 2000.0);  
        double d = s.placa();  
        System.out.println("Wynagrodzenie: " + d);  
    }  
}
```


```
run:  
Wynagrodzenie dla pracownika Kowalski  
wynosi 2000.0  
Wynagrodzenie: 2000.0
```

# Słowo kluczowe *final* zapobiega dziedziczeniu

Słowo kluczowe *final* wyłącza możliwość przysłonięcia metody. Podklasy nie będą mogły zaimplementować własnej metody np..

```
class KlasaA{
    protected double x, y;
    KlasaA() {}
    KlasaA(double x, double y) {
        this.x = x;
        this.y = y;
    }
    final void drukuj () {
        System.out.println("x = " + x);
        System.out.println("y = " + y);
    }
}
```

# Klasa Object

Klasa Object jest klasą bazową dla wszystkich klas w języku Java i definiuje metody dostępne we wszystkich tworzonych obiektach.

| Metoda                            | Opis                                                              |
|-----------------------------------|-------------------------------------------------------------------|
| Object Clone()                    | Tworzy obiekt będący obiektem przekazanego typu.                  |
| boolean equals<br>(object object) | Sprawdza identyczność przekazanego obiektu.                       |
| void finalize()                   | Metoda wywołana przed usunięciem nieużywanych obiektów z pamięci. |
| Class getClass()                  | Pobiera klasę obiektu                                             |
| String toString()                 | Zwraca łańcuch opisujący obiekt.                                  |
| int hashCode()                    | Zwraca skrót kodu obiektu (ang. hash code)                        |

# Interfejsy

Interfejs jest opisem, co klasa powinna robić, ale nie zawiera opisu tego w jaki ma to wykonywać. Jest to zbiór wymagań dotyczących klas, które chcą dostosować się do interfejsu. Rozważmy przykład wykorzystania metody *sort()* z **Arrays** umożliwiającej sortowanie tablice obiektów implementującej interfejs **Comparable**. Postać interfejsu **Comparable** jest następująca:

```
public interface Comparable{  
    int compareTo( Object ob)  
}
```

Oznacza to, że każda klasa implementująca interfejs **Comparable** musi posiadać metodę *compareTo* oraz że metoda ta przyjmuje argument typu **Object** i zwraca liczbę typu **integer**.

Założmy teraz, że chcemy teraz użyć metody *sort()* z klasy **Arrays**, by posortować tablicę obiektów pracowników. Klasa Pracownik musi zaimplementować interfejs Comparable. W deklaracji klasy pracownik musi się pojawić informacja o implementacji interfejsu za pomocą słowa kluczowego **implements** i deklaracja klasy ma postać:

```
class Pracownik implements Comparable
```

Następnie klasa pracownik powinna zaimplementować metodę *compareTo*. Metoda *compareTo* interfejsu Comparable zwraca liczbę typu integer. Jeśli obiekty są równe zwracane jest zero i w przypadku, gdy obiekty nie są równe liczbę dodatnią albo ujemną w zależności od wyniku porównania obiektów. Przyjmijmy teraz, że chcemy uporządkować pracowników w zależności od numeru id w odwrotnej kolejności. Postać metody może wyglądać następująco:

```
public int compareTo(Object obj) {  
    Pracownik e = (Pracownik) obj ;  
    if( id > e.id) return -1;  
    else  
        if( id < e.id) return 1;  
    else  
        return 0;  
}
```



run:



4 Żyrafa +48506231456



3 Lewandowska +3378767243456



2 Alen +48608567234

1 Zamojski +495568989098

Ten konkretny interfejs posiada tylko jedną metodę. W ogólnym przypadku interfejs może zawierać dowolną liczbę metod. Metody nigdy nie są implementowane wewnątrz interfejsu. Metody są implementowane wewnątrz klas dostępujących się do wymagań interfejsu i zawsze muszą być dekladowane jako publiczne.

## Interfejs

Interfejs jest definicją abstrakcyjnego typu posiadającego jedynie operacje, a nie dane. W konkretnej klasie implementowane są wszystkie metody interfejsu w celu udostępnienia operacji wykonywanych przez klasę bazową. W języku Java przykładowy interfejs i jego realizacja może wyglądać następująco:

# Przykład interfejsu w języku Java

```
interface Interfejs{  
    public void metoda();  
}  
class Klasa implements Interfejs{  
    @Override  
    public void metoda(){  
        // ...  
    }  
}
```

Przedstawmy przykład wykorzystania interfejsu Comprable klasy Arrays przez klasę pracownik w celu uporządkowania rekordów wg. pola id malejąco (ang. ascending).

```
package c4n;
import java.util.*;
class Pracownik implements Comparable {
    private int id;
    private String nazwisko, telefon;
// Pola klasy.
    public Pracownik() { }
// Konstruktor domyślny.
    public Pracownik(int id, String nazwisko,
                     String telefon){
// Przeciążenie konstruktora.
        this.id = id;
        this.nazwisko = nazwisko;
        this.telefon = telefon;
    }
}
```

```
    public String toString() {  
        // Przeciążenie metody ToString().  
        return String.format  
            ("%d %s %s", id, nazwisko,  
                telefon);  
    }  
    public int compareTo(Object obj) {  
        Pracownik e = (Pracownik) obj ;  
        if( id > e.id) return -1;  
        else  
            if( id < e.id) return 1;  
        else  
            return 0;  
    }  
}
```

```
public class C4n {  
    public static void main(String[] args) {  
        Pracownik[] a = new Pracownik[4];  
        // Konstrukcja tablicy na zarządzanym stosie.  
        a[0] = new Pracownik  
            (4, "Żyrafa", "+48506231456");  
        a[1] = new Pracownik  
            (1, "Zamojski", "+495568989098");  
        a[2] = new Pracownik  
            (3, "Lewandowska", "+3378767243456");  
        a[3] = new Pracownik  
            (2, "Alen", "+48608567234");  
        // Wpisanie rekordów.  
        Arrays.sort(a);  
        for (Pracownik c: a)  
            System.out.println(c);  
        // Wydruk zawartości tablicy.  
    }  
}
```

Uporządkowanie rekordów klasy pracownik wg. pola id malejąco za pomocą metody sort() wykorzystującej interfejs Comparable klasy Arrays



```
run:
```

```
4 Żyrafa +48506231456
```

```
3 Lewandowska +3378767243456
```

```
2 Alen +48608567234
```

```
1 Zamojski +495568989098
```

# Interfejsy

Interfejs definiuje klasę i jej elementy bez określania ich dokładnych implementacji. Interfejs określa, co klasa ma rozbić, ale nie wskazuje, w jaki ma to robić. Interfejsy są składowo podobne do klas, ale nie mogą w nim pojawiać zmienne składowe a metody deklaruje się bez podawania jakiegokolwiek kodu. W ten sposób określa się metody, które klasa musi zaimplementować, ale nie wskazuje się jak ma tego dokonać. Dowolna liczba klas może implementować dany interfejs. Dowolna klasa może implementować wiele interfejsów. Interfejs spełnia następujące trzy warunki:

- Definicja interfejsu – interfejs definiuje składowe klasy, takie jak metody i inne, jednak sam nie udostępnia implementacji tych elementów.
- Klasa implementująca interfejs – klasa implementująca pewien interfejs tworzy implementację wszystkich metod zdefiniowanych w tym interfejsie. Klasa może je implementować w dowolny sposób.
- Klienci – obiekty tworzące egzemplarze klasy implementującej interfejs znane są pod nazwą klientów. Klient wywołuje metody zdefiniowane w interfejsie i wykorzystuje ich implementację w określonej klasie.

Różnice między interfejsem a abstrakcyjną klasą bazową.

- Klasa abstrakcyjna może zawierać zaimplementowane metody abstrakcyjne (przynajmniej jedną), natomiast interfejs nie może zawierać żadnych implementacji metod.
- Klasa abstrakcyjna może zawierać konstruktory i destruktory, które nie mogą pojawiać się w interfejsie.
- Klasa może implementować wiele różnych interfejsów, ale dziedziczyć może tylko po jednej klasie bazowej.

# Definicja interfejsu

Interfejs definiuje się podobnie jak klasę i ogólna postać takiej definicji:

```
dostęp interface nazwa {  
    typ zmienna-finalna1 = wartość;  
    //...  
    typ zmienna-finalnaN = wartość;  
    zwracany – typ nazwa-metody1(lista – parametrów);  
    //...  
    zwracany – typ nazwa-metodyM(lista – parametrów);  
}
```

Element *dostęp* jest typu *public* lub nie jest stosowany wcale. Gdy pominie się specyfikację dostępu, interfejs będzie dostępny, jedynie dla klas tego samego pakietu. Dla dostępu *public* interfejs będzie dostępny dla wszystkich innych klas.

Wewnątrz deklaracji interfejsu mogą pojawiać zmienne. Są one jednak niejawne ustawione na zmienne typu *final* i *static*, co oznacza, że nie mogą być zmienione przez implementującą go klasę i trzeba je zainicjować. Wszystkie metody i zmienne są niejawnie ustawione na typ *public*, jeśli sam interfejs został zadeklarowany z użyciem słowa kluczowego *public*.

Poniżej znajduje się przykład interfejsu. Deklaruje on interfejs, który zawiera metodę o nazwie `callBack()` przyjmującą jeden parametr typu `int`.

# Definiowanie interfejsu

```
package p6;  
interface Callback {  
    void callBack(int param);  
}
```

```
package p6;  
public class Client implements Callback{  
    public void callBack(int p){  
        System.out.println("Wywołanie metody callBack() "+p);  
    }  
}
```

```
package p6;  
public class AnotherClient implements Callback {  
    public void callBack(int p){  
        System.out.println("Inna wersja metody callBack() " + p*p);  
    }  
    void nonIfaceMeth(){  
        System.out.println("klasa implementująca interface");  
        System.out.println("może zawierać także własne metody");  
    }  
}
```

```
package p6;
public class P6 {
    public static void main(String[] args) {
        Callback c = new Client();
        AnotherClient ob = new AnotherClient();
        c.callBack(42);
        c = ob;
        c.callBack(42);
    }
}
```

| Wyjście - P6 (run) X                                                                |                                          |
|-------------------------------------------------------------------------------------|------------------------------------------|
|    | run:                                     |
|    | Wywołanie metody callBack() 42           |
|   | Inna wersja metody callBack() 1764       |
|  | BUILD SUCCESSFUL (total time: 0 seconds) |

# Zmienne w interfejsie

Interfejsów można używać do wprowadzania w wielu klasach tych samych stałych. Wystarczy w interfejsie zadeklarować i zainicjować zmienne. Klasa implementująca taki interfejs będzie w swoim zasięgu widziała wszystkie te zmienne jako stałe (przypomina to tworzenie stałych dyrektywą *#define* lub deklaracją *const* w języku C lub C++). Jeśli interfejs nie zawiera żadnych metod, wtedy implementująca go klasa nie musi deklarować żadnych dodatkowych metod. Zmienne występujące w interfejsie są widoczne przez klasę korzystającą z interfejsu jako *final*. Kolejny przykład używa interfejsu do zaimplementowania automatu podejmującego decyzję.

```
package p7;
interface SharedConstants {
    int NO = 0;
    int YES = 1;
    int MAYBE = 2;
    int LATER = 3;
    int SOON = 4;
    int NEVER = 5;
}
```

```
package p7;
import java.util.Random;
public class Question implements SharedConstants {
    Random rand = new Random();
    int ask() {
        int prob = (int) (100*rand.nextDouble());
        if(prob < 30) return NO;
        else
            if(prob < 60) return YES;
        else
            if(prob < 70) return MAYBE;
        else
            if(prob < 85) return SOON;
        else
            return NEVER;
    }
}
```

```
package p7;
public class AskMe implements SharedConstants{
    static void answer(int result){
        switch(result){
            case NO:
                System.out.println("Nie");
                break;
            case YES:
                System.out.println("Tak");
                break;
            case MAYBE:
                System.out.println("Może");
                break;
            case LATER:
                System.out.println("Później");
                break;
            case SOON:
                System.out.println("Wkrótce");
                break;
            case NEVER:
                System.out.println("Nigdy");
                break;
        }
    }
}
```

```
package p7;

public class P7 {

    public static void main(String[] args) {
        Question q = new Question();
        AskMe.answer(q.ask());
        AskMe.answer(q.ask());
        AskMe.answer(q.ask());
        AskMe.answer(q.ask());
    }
}
```

| Wyjście - P7 (run) ×                                                                |         |
|-------------------------------------------------------------------------------------|---------|
|    | run:    |
|    | Tak     |
|    | Wkrótce |
|  | Tak     |
|                                                                                     | Wkrótce |
| BUILD SUCCESSFUL (total time: 0 seconds)                                            |         |

| Wyjście - P7 (run) ×                                                                  |         |
|---------------------------------------------------------------------------------------|---------|
|    | run:    |
|    | Wkrótce |
|    | Nie     |
|  | Tak     |
|                                                                                       | Nigdy   |
| BUILD SUCCESSFUL (total time: 0 seconds)                                              |         |